



A job search engine you operate by talking to it. It read 19,550 postings and surfaced 161.

CareerKit / open source, MIT
github.com/omoji-personal/careerkit

Finding job postings was never the hard part. The hard part is that almost everything you find is noise, and the few that matter are buried by the time you reach them. CareerKit polls employer applicant tracking systems directly, scores every posting against rules you wrote yourself, and hands you the short list with its reasoning attached.

It is built for people who do not write code. You talk to Claude Code; Claude runs the machinery.

SPECIFICATION			
ATS platforms polled	17	Public job feeds	14
Auto-discoverable	13 platforms	One run	19,550 read, 161 passed
Regression tests	157	Continuous integration	3.10 / 3.12 / 3.13
Engine	Python, SQLite	Licence	MIT

1	What it does
2	Setup, and why the interview is not optional
3	How scoring works: lanes, rails, gates
4	The daily loop
5	Calibration: keeping the filter honest
6	What leaves your machine
7	Failure modes worth knowing
8	Reference

What it does

CareerKit watches the job boards employers actually post on, rather than the search pages that aggregate them. That distinction is most of the value: an employer's own applicant tracking system lists the requisition the day it opens, with its real title, location and requisition id, and it stops listing it the day it closes.

Every posting it collects is scored against your rules. The output is a report, ordered, with the reasoning attached to each entry, so you can tell the difference between a role that matched and a role that merely appeared.

The ratio is the product

In a single run against 273 employer boards and 14 feeds, CareerKit pulled 19,550 postings. 275 were in the right job family at all, and of those **161** cleared every screenable rail. Everything else was screened out against a named rule, and `audit` shows you those decisions whenever you want to argue with them.

WHY THIS MATTERS

A filter you cannot inspect is a filter you cannot trust. Screened-out postings are not kept in the database, so `audit` re-polls the boards and re-scores them to show you what was killed and why. That is the command that lets you overrule your own gates.

What it is not

It is not a mass-application tool. It fills a form completely and stops at the submit button, because the certification on an application is yours to make. It does not create accounts, handle passwords, or answer "prove you are human" checks.

Setup, and why the interview is not optional

You need Claude Code (Pro plan minimum), Python 3.10 or newer, and git. Install Claude Code with `npm install -g @anthropic-ai/claude-code`, which needs Node 18 or newer. Chrome and the Claude-in-Chrome extension are optional, and only needed if you want it filling application forms for you. macOS and Linux work as they are; on Windows, run the shell scripts from Git Bash or WSL.

```
git clone https://github.com/omoji-personal/careerkit.git
cd careerkit
./setup.sh          # checks prerequisites, builds .venv, installs deps
claude              # open Claude Code in this folder
/setup              # about 40 minutes
```

`/setup` interviews you: what you do, what you want next, where you will work, what you will not do, and what is true about you. It writes `profile/profile.yaml` and `profile/claims.md`.

It is not skippable. Until that file exists the engine has no rules to score against, and it will tell you so rather than guess. A tool that guesses your criteria produces a report that looks exactly as confident as a correct one.

The claims register

`profile/claims.md` is the list of things that may be said about you. Nothing goes into a resume, cover letter, form answer or outreach message unless it is in there. When you tell Claude something new about yourself, it gets added, with your confirmation, before it is used.

How scoring works: lanes, rails, gates

Lanes are the job families you want, each with its own titles and weight. **Rails** are the hard constraints: location, compensation floor, clearance, product specialisations you do not hold, role families you rule out. **Gates** are the four possible verdicts.

VERDICT	MEANING	WHAT TO DO
QUALIFIED	Passes every screenable rail, with evidence found in the posting.	Read it. This is the short list.
VERIFY	Looks right, but one rail could not be evidenced from the text.	Check the one open question yourself.
SLOT-BLOCKED	Clears the rails, but the role family is one you rule out.	Nothing, unless your criteria changed.
EXCLUDED	Fails a hard rail. Never surfaces in the report.	Review in bulk with audit .

Your rules are the only rules

`profile/profile.yaml` is the single place scoring rules exist. To change what surfaces, change that file, usually by asking Claude to run `/criteria`. Never by editing engine code.

This is enforced, not merely intended: a test parses every module in `engine/` and fails the build if one of a list of person-specific terms appears outside a docstring. It is a tripwire rather than a proof, but it means the engine cannot quietly acquire an opinion about what you want.

IF A RULE IS UNUSABLE

The engine prints what it ignored and carries on, except in exclusion lists, where an uncompileable rule raises instead. Dropping an exclusion fails open: the rail disappears and everything you banned starts surfacing, silently.

The daily loop

<code>/search</code>	Sweep every board and feed, report what is genuinely new since the last run.
<code>/evaluate <url></code>	An honest fit read on any posting, from anywhere. Names the real disqualifiers.
<code>/apply <url></code>	Build the application pack and fill the form. You review and submit.
<code>/prep <company></code>	Interview preparation from your own story bank.
<code>/track</code>	Where everything stands.
<code>/ingest <url></code>	Score and register anything you found elsewhere, so the engine knows about it.
<code>/compose</code>	Resume, cover letter and profile work, gated by your claims register.
<code>/outreach</code>	Referrals, thank-yous and nudges. Draft only by default.
<code>/criteria</code>	Change what surfaces.
<code>/audit</code>	Review what the gates killed. Run it monthly.

Running the engine directly

Claude does this for you, but it is an ordinary command line tool.

```
./careerkit.py pull           # poll everything, score, write a report
./careerkit.py doctor        # profile, sources, freshness, drift: one
check
./careerkit.py status        # what is in the database, which sources
are broken
./careerkit.py audit         # what the gates killed, and why
./careerkit.py report        # rebuild the report ( --format json|csv
to export )
./careerkit.py history       # every status change, in order
./careerkit.py claims-lint out/cover.md
./careerkit.py mark UID applied
./careerkit.py ingest-url -- URL # register one posting you found
elsewhere
./careerkit.py ingest-urls FILE # ... or a file of them
./careerkit.py discover NAME  # find a company's board and
register it
./careerkit.py verify        # confirm each board belongs to the
right company
./careerkit.py db check|backup # integrity check, manual backup
```

Calibration: keeping the filter honest

A screening tool drifts toward the mistake you cannot see. Rails that are slightly too tight remove real matches, and because you never see what was removed, the reports keep looking reasonable. This is the failure that costs the most and announces itself the least.

audit is the answer. It re-scores and shows you every kill with its reason, grouped, with examples. You argue with it, and what you disagree with becomes a profile change.

RUN IT MONTHLY

A filter nobody reviews is a filter nobody can trust. Thirty minutes of arguing with your own gates is worth more than any amount of additional sourcing.

Zero results is a claim

"Nothing new this week" can mean nothing was posted, or it can mean a board changed its API and has been returning an error for eleven days. CareerKit tracks the health of every source separately, reports consecutive failures, and flags a source that returned nothing when it had postings last run. A quiet week has to earn the description.

What leaves your machine

Worth being precise about, because "nothing" would be untrue.

- **To Anthropic:** everything Claude reads. Your resume, your claims register, postings, your interview answers. Governed by your own Claude plan's data terms, the same as any other conversation.
- **To job boards and feeds:** ordinary outbound web requests from your machine, throttled to one request per host every 0.7 seconds. Your identity is not attached to them.
- **One exception:** the USAJobs feed requires your registered email in a request header. That is their API's rule. Skip the feed if you would rather not.
- **No telemetry.** Nothing is reported to the author or anyone else.

Delete profile/, data/ and out/ and nothing personal remains locally.

On scraping

Every feed declares what it is: an official public API, one that needs your own key, or a scraper. Two read public search pages rather than official APIs. They are off by default, can be rate limited or blocked, and their terms of use are yours to read. Any run whose results include a scraper says so in the report.

Failure modes worth knowing

A job-search engine that is subtly wrong does not crash. It hands you a clean, confident report with the wrong jobs in it. Most of the engineering here is aimed at that. Some of what it now defends against, all of it found in this code:

- Two different openings with the same title at one employer collapsing into one entry, so marking one applied silently hid the other.
- A blank entry in an exclusions list compiling to a pattern that matched every posting ever written. Zero results is indistinguishable from a quiet day.
- Postings that had closed continuing to surface as live, because nothing wrote back that they were gone.
- A stipend of \$500 beside a real salary band being read as \$500,000 and becoming the top of the range.
- "New since last time" meaning "first seen today", so two runs in one day announced the same roles twice.
- A consistency check whose pattern matched nothing, so it reported no problems forever. A check that cannot fail is worse than no check.

THE TEST SUITE IS THE POINT

The suite is written against defects that were really in the code rather than for coverage, and it runs 162 cases across 122 test functions (several are parametrised over the inputs that actually broke). When a batch of fixes went in, the fixes themselves were attacked, and every serious defect in the second round had been introduced by a fix from the first.

Reference

Layout

profile/	Yours. Profile, claims, style, employer registry, tracker. Gitignored.
data/	The SQLite database and HTTP cache. Gitignored.
out/	Generated reports, one per run, plus an immutable copy of each. Gitignored.
engine/	Adapters, feeds, scoring, storage, reporting. No personal data, ever.
.claude/skills/	The workflows Claude follows for each command.

Sharing it

`./new-instance.sh <name>` clones a fully separate instance with its own profile, database and reports. Instances never see each other's data. Use one per person. Instances clone from the public repository, so `git pull` inside one really does bring engine fixes.

Licence

MIT. Use it, fork it, change it. If you improve the engine, the improvement belongs in the template so every instance inherits it.

| | | | | |

github.com/omoji-personal/careerkit